

Summer Undergraduate Internship Final Report

UC Davis 2026

Isabella Mo

August 23, 2026



Introduction

I am an undergraduate student at the University of California, Davis, pursuing a Bachelor of Science in Electrical Engineering with a focus on analog and digital systems. As the Electrical Subteam Lead for Cyclone RoboSub, I oversee the design and integration of custom printed circuit boards for internationally competitive autonomous underwater vehicles. Complementing my hardware design experience, I work at the Center for Nano-Micro Manufacturing (CNM2), where I engage with advanced electronics fabrication and cleanroom manufacturing processes.

As I continue to deepen my involvement in hardware development, I am enthusiastic about inspiring others to explore the field. This summer, I guided students through photolithography processes during educational workshops hosted at CNM2. During the academic year, I organize events for the Institute of Electrical and Electronics Engineers (IEEE) Student Branch in collaboration with the Student Startup Center to bridge engineering and entrepreneurship.

My research foundation is rooted in my contributions as an Undergraduate Research Assistant in Professor Rajeevan Amirtharajah's Micropower Circuits and Systems Group. We collaborate with members of Professor Barbara Linke's Manufacturing and Sustainable Technologies Research (MASTeR) Lab and Professor Camli Badrya's Davis Applied Aerodynamics Lab (DAAL) at Next Level Research (NLR) Green Aviation to explore innovations in zero-emission aircraft lifting surfaces. Through the BE-STARSE Program, I sought to support our work by leveraging low-power digital design to enhance sensor data analysis.

Scope of Work

NLR Green Aviation is an initiative led by members of Micropower Circuits and Systems Group, MASTeR Lab, and DAAL that seeks to study airflow transitions across airfoil surfaces using Micro-Electro-Mechanical System (MEMS) microphones. A custom digital filter was developed in Verilog to reduce noise recorded by these sensors during data collection operations at the Aeronautical Wind Tunnel. The application aims to help members of NLR Green Aviation make better-informed recommendations for optimizing airflow efficiency using bio-inspired lifting-surface geometries. The following are core research deliverables.

Objective 1: Design Requirements Specification

Experimental procedures were designed and executed with members of NLR Green Aviation to capture acoustic airflow data in the wind tunnel. Technical requirements for the filter were defined by using audio recordings from an analog MEMS microphone.

Objective 2: Standard Filter Model

A standard bandpass filter was identified for isolating airflow transition signals from background noise. It was simulated using the MATLAB Filter Designer application and a custom script to validate the design by processing theoretical signals and raw acoustic data.

Objective 3: Optimized Filter Model

An optimized version of the filter was designed in Verilog to explore a power- and resource-efficient hardware alternative. Reviews of research papers and textbook references provided by Professor Amirtharajah were conducted to identify component features and integration techniques. Subsequently, a block diagram and logic flowchart were developed to map the complete architecture. Foundational parts of the system were designed on Quartus Prime and verified using ModelSim. Then, they were assembled and further simulated to evaluate the quality of signal processing capabilities.

Objective 4: Design Comparison

The standard and optimized filters were evaluated using Quartus analysis tools to compare power and resource trade-offs between the designs.

Background

Real-world signals often differ from the theoretical models used to understand them due to non-ideal effects, which must be taken into consideration when interpreting results. Today, numerous technologies exist to neutralize these variations, allowing us to spend more time and resources studying purer forms of signals. A Finite Impulse Response (FIR) filter is one solution. The digital system removes the contributions of select frequencies from a dataset, taking a weighted average of current and past input values to produce a signal most influenced by the passband. Not relying on internal feedback allows it to process signals and settle to zero within a finite period, making it a reliable type of implementation.

The desire for a signal-processing filter arose after members of NLR Green Aviation identified significant background acoustic signals in the MEMS microphone data from the wind tunnel facility they were testing to determine whether these sensors could be sufficient for their research. An FIR filter was selected to ensure system stability and a linear phase response, which is critical for preserving the characteristics of the airflow transition in the acoustic waveforms. The computational structure for processing data points was modified to use Distributed Arithmetic (DA) units that rely on Look-Up Table (LUT) searches rather than traditional Multiply-Accumulate (MAC) blocks, meeting power and resource optimization goals that align with overarching research efforts to develop sustainable aircraft designs.

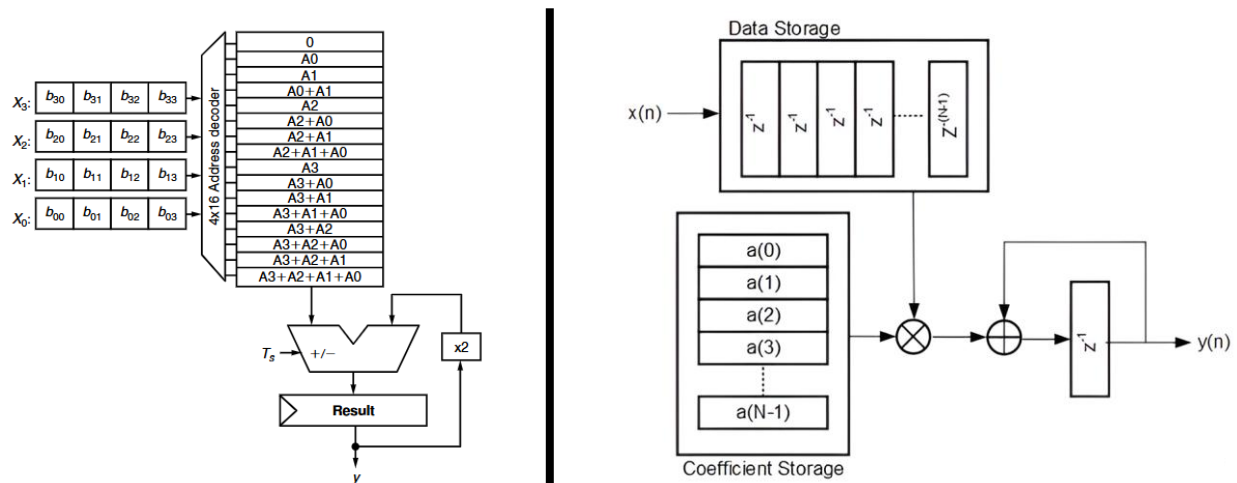


Figure 1. DA [1] and MAC [2] Architecture Comparison for an FIR Filter.

Standard FIR Filter

Parameters for the filter were derived from spectral analysis of wind tunnel datasets collected by an IMP23ABSU analog MEMS microphone recording airflow across the surface of an airfoil at 60 mph and a 0° angle of attack, the baseline data that NLR Green Aviation researchers used to study resonant frequencies. Discussions with researchers from the Micropower Circuits and Systems Group identified a targeted passband of 10–13 kHz after applying signal-processing techniques in Audacity, a digital audio workstation.

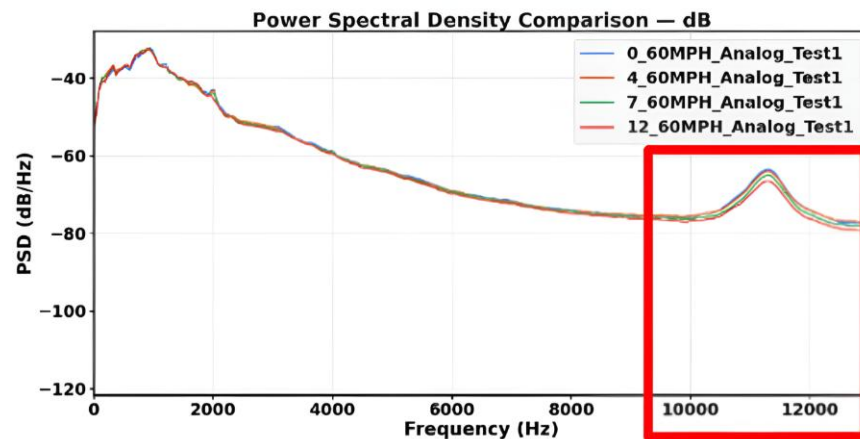


Figure 2. Power Spectral Density Graph [3] of Analog MEMS Microphone Recording. The Red Box Indicates the Targeted Frequency Range.

A bandpass FIR filter featuring a 192 kHz sampling frequency was modeled using the MATLAB Filter Designer application. To ensure accurate rejection of frequencies above and below the passband, stopband attenuation was set to 60 decibels.

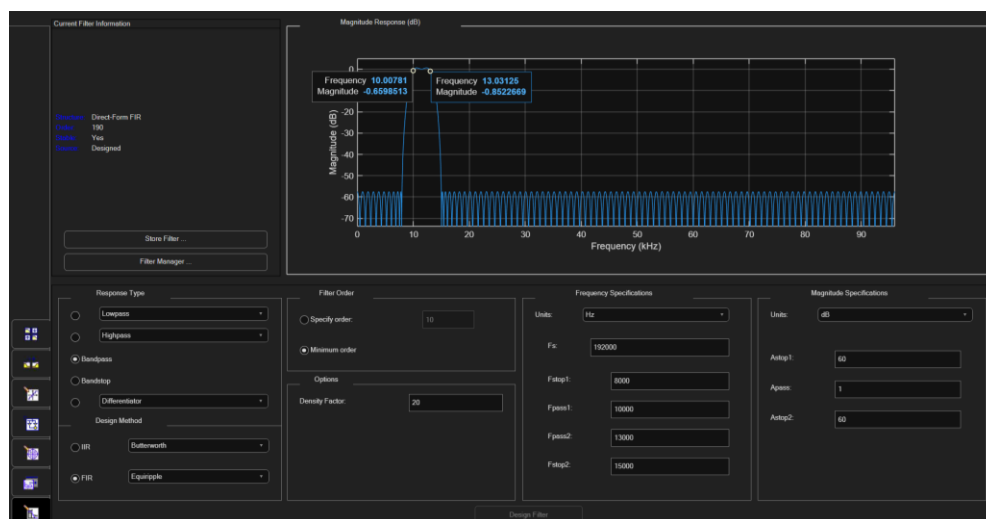


Figure 3. Settings for the Bandpass FIR Filter in the MATLAB Filter Designer Application.

The application generated 191 custom coefficients, which were used to develop a MATLAB script for a standard bandpass FIR filter to verify that this type of model could meet design requirements. To process the raw data from the audio recordings, it was quantized from a floating-point range to a signed 16-bit integer range to match the microphone's bits per sample.

The model was validated by combining 500 Hz and 11.5 kHz sine waves. Graphs of the raw and filtered waveforms and their Power Spectral Densities (PSD) were generated in MATLAB, demonstrating successful isolation of the high-frequency component.

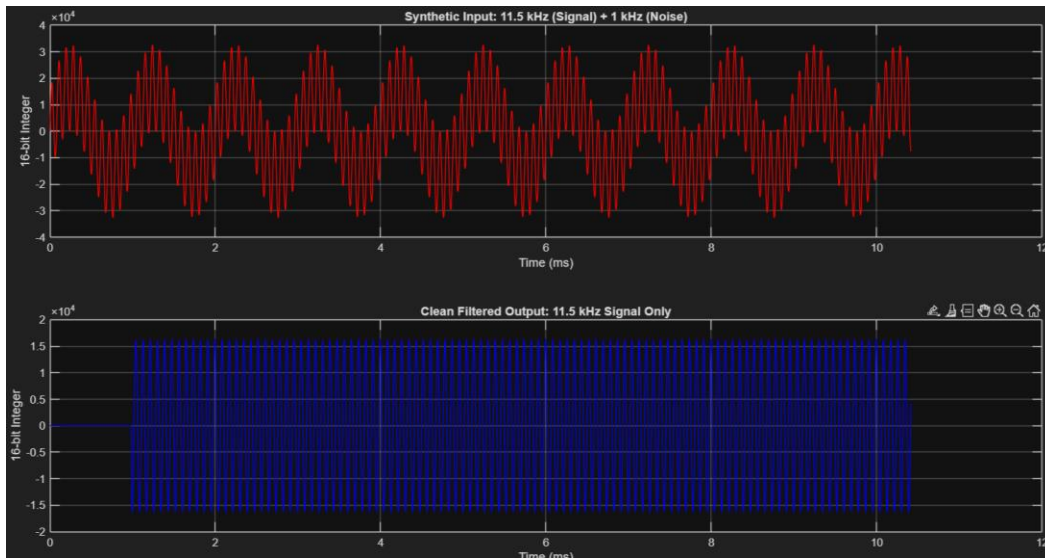


Figure 4. Time Domain Plots of Input Sine Wave and Filtered Signal.

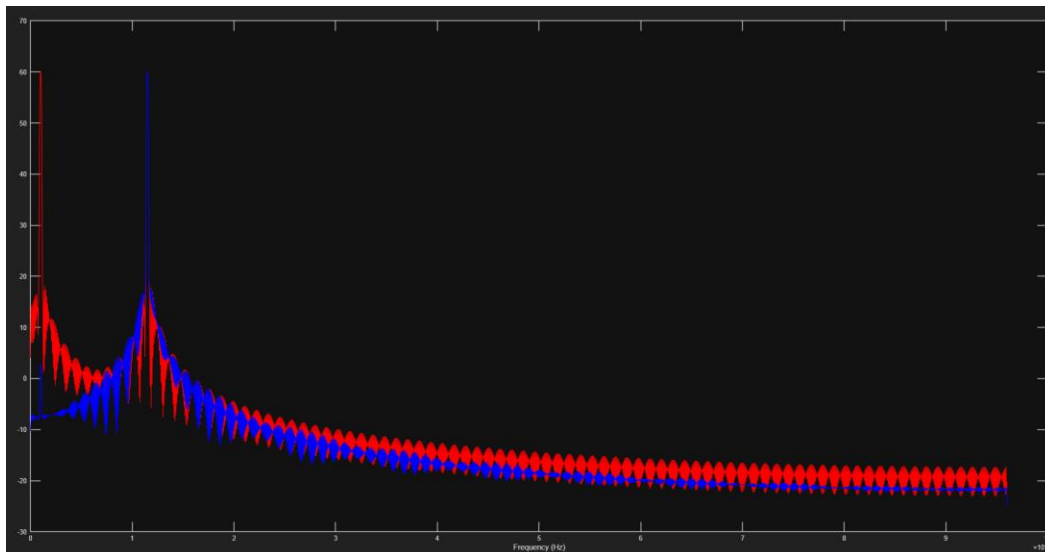


Figure 5. PSD of Input Sine Wave and Filtered Signal.

Raw acoustic data was then filtered to determine the capabilities of using the standard design for aeroacoustic applications. The same type of graphs were used to visualize the results.

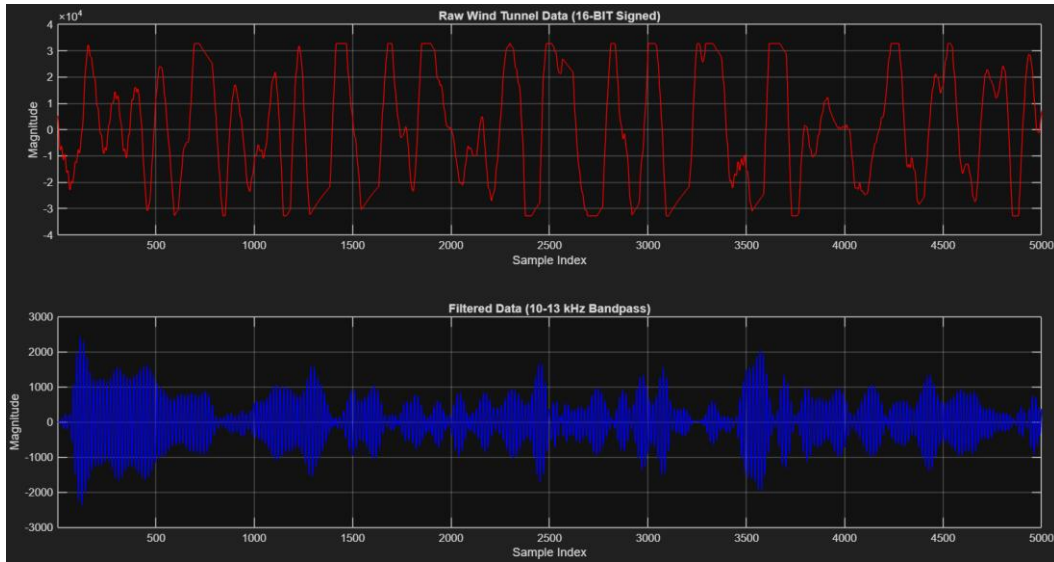


Figure 6. Time Domain Plots of Raw Wind Tunnel Data and Filtered Data.

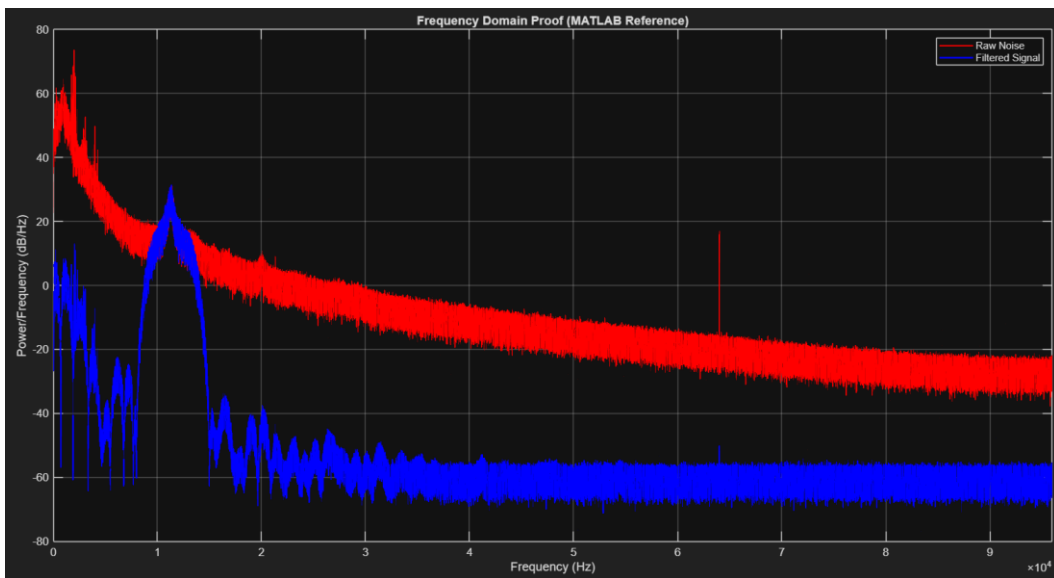


Figure 7. PSD of Raw Wind Tunnel Data and Filtered Data.

The raw signal possessed noticeable clipping, which initially concerned NLR Green Aviation researchers, as it could indicate the analog MEMS microphone was an insufficient choice if it could not tolerate high-decibel airflow. However, digital signal processing techniques were used to remove signals irrelevant to airflow transitions, producing new, undistorted filtered data. The standard model yielded consistent results, so it would be reasonable to move forward with developing an optimized version. The standard model served as the functional benchmark referenced when verifying the accuracy of the optimized design.

Optimized FIR Filter

An optimized bandpass FIR filter was sought to process 16-bit acoustic data points while achieving power and resource efficiency. The design process included the development of logic flow, hardware architecture, component- and system-level design, and simulation.

Core Process

An iterative loop is the key component driving the FIR filter. A 16-bit data point is filtered one bit at a time through parallel Look-Up Table (LUT) searches and an adder tree, with the new data point reconstructed over 16 clock cycles.

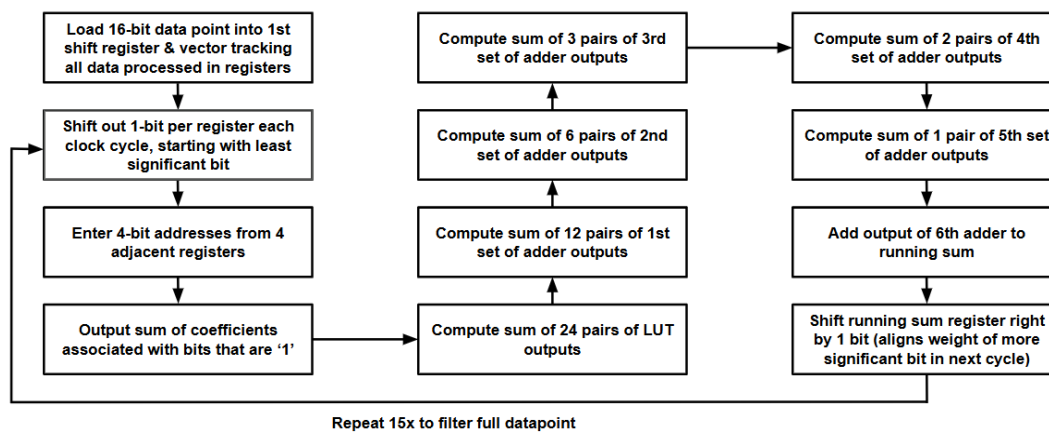


Figure 8. Flowchart of FIR Filter Computational Logic to Process One 16-Bit Data Point.

Hardware Architecture

The process is initiated by loading a 16-bit data point into the first 16-bit Parallel-In, Serial/Parallel-Out (PISPO) Shift Register, and the remaining ones are loaded with any previous data points processed while the filter is active. Additionally, the value is stored in a vector that stores each data point entered to load the remaining PISPO Shift Registers simultaneously. The system extracts one bit of each data point per clock cycle, starting with the Least Significant Bit (LSB) and ending with the Most Significant Bit (MSB). Outputs from 4 adjacent shift registers are combined into a 4-bit address, which a 4x16 address decoder uses to identify the precomputed linear combinations of four coefficients for the filter: the sum of coefficients corresponding to the address bits with binary values of '1'.

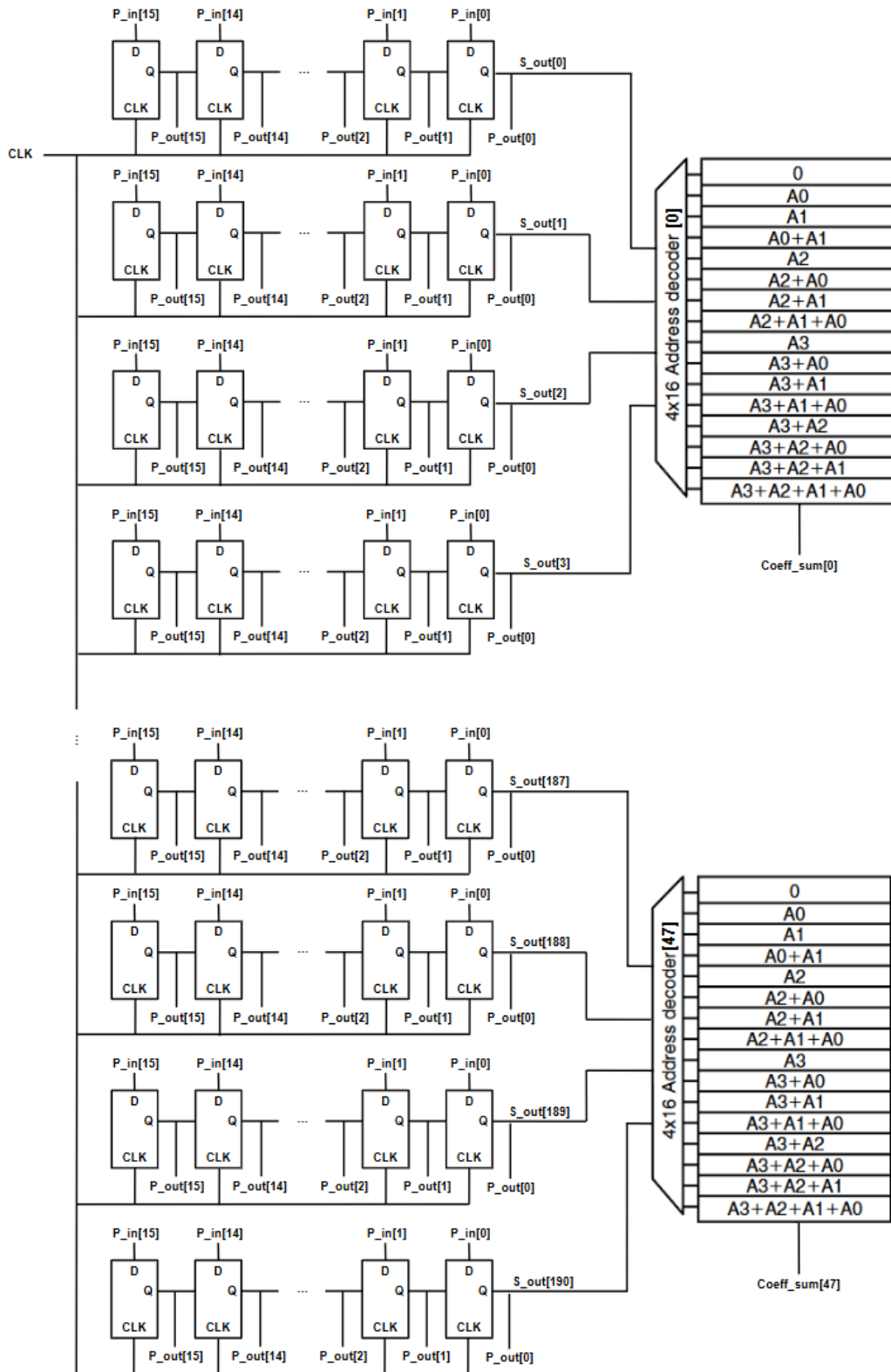


Figure 9. Block Diagram of 191 16-Bit Parallel-In, Serial/Parallel-Out Shift Registers and 48 Look-up Tables [4] for the FIR Filter Structure.

The parallel 4-bit LUT blocks output 48 coefficient sums simultaneously, so a multi-layer binary adder tree combines them into a unified value. Computation is routed sequentially through six distinct layers of adders, first summing 24 pairs of raw LUT outputs, and then combining those output pairs until the specific weight of the bit processed during that cycle is computed.

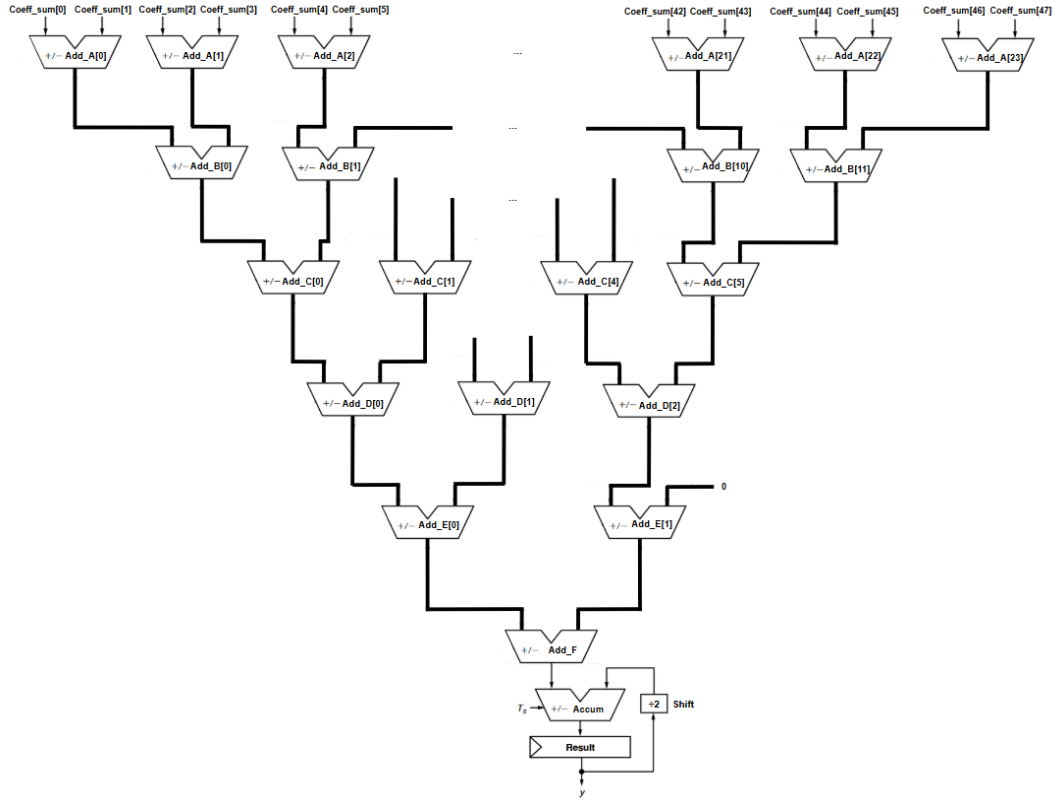


Figure 10. Block Diagram of a 6-Level Adder Tree Using 48 2-Input Adders and an Accumulator for the Final Partial Product of the 16-Bit Datapoint [4].

The output of the final adder is entered directly into the Accumulator, a running-sum register. To account for the binary weight of the data bits, which starts with the LSB, the Accumulator is shifted right by 1 bit at the end of each bit-processing cycle. The sequence illustrated by the logic flowchart is repeated 15 more times within the hardware architecture until all 16 bits of the data point are fully processed, yielding the final filtered result.

Filter Components

A component-level approach was taken to draft the optimized filter in Verilog, leading to a high-level understanding of the system and its foundational components. Each part was validated using testbenches run in ModelSim.

The 16-bit PISPO Shift Register is composed of D Flip-Flops arranged in series, configured to store a binary word representing a raw acoustic data point. It can output the entire word at once, which is used to store the input in the vector tracking the stream of data points. Additionally, it can shift out one bit at a time on each clock cycle, starting with the LSB, which aligns with the configuration the Accumulator uses to construct the filtered data point: process the magnitude bits, then the sign bit.

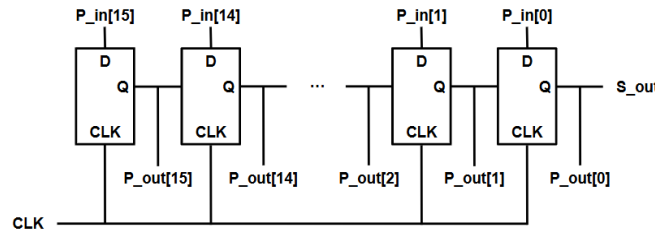


Figure 11. Block Diagram of 16-Bit Parallel-In, Serial/Parallel-Out Shift Register.

In Verilog, a 16-bit vector is used to identify which bit of a raw data point is to be output during a clock cycle. Additionally, enable signals control when loading and shifting tasks delegated to the shift register are executed, such as loading the 16-bit value before outputting bits.

```

1 module PISPO_Register(
2     input signed [15:0] P_in,          // 16-bit data point entering
3     input CLK,                        // clock signal for timing shifting
4     input RST,                        // tied to global reset signal in top-level
5     input load_en,                    // load enable signal
6     input shift_en,                   // shift enable signal
7     output S_out,                     // 1-bit serial output exiting
8     output reg signed [15:0] P_out    // 16-bit parallel output
9 );
10
11 reg signed [15:0] shift_reg; // 16-bit word used to shift through sequence of data point bits
12
13 always @(posedge CLK) begin
14     if (RST == 1'b1) begin // wipe values stored in output variables
15         shift_reg <= 16'sd0;
16         P_out <= 16'sd0;
17     end
18     else if (load_en == 1'b1) begin // load new word
19         shift_reg <= P_in; // pass out to get shifted to next shift register in sequence (i+1)
20         P_out <= P_in;
21     end
22     else if (shift_en == 1'b1) begin // shift bit sequence to the right (preserves sign bit at shift_reg[15])
23         shift_reg <= {shift_reg[15], shift_reg[15:1]};
24     end
25 end
26
27 assign S_out = shift_reg[0]; // order bits are outputted: LSB to MSB
28
29 endmodule

```

Figure 12. Verilog Code for 16-Bit Parallel-In, Serial/Parallel-Out Shift Register.

A simulation was performed in ModelSim to verify the module. The initial state of the component was configured as a 16-bit word of zeros with the output in a low state. When loading was enabled, a 16-bit word of ones and zeros was loaded, and its LSB was output within the same clock cycle. After the enable signal was toggled low, the signal for shifting bits out of the shift register on each new clock cycle was asserted.

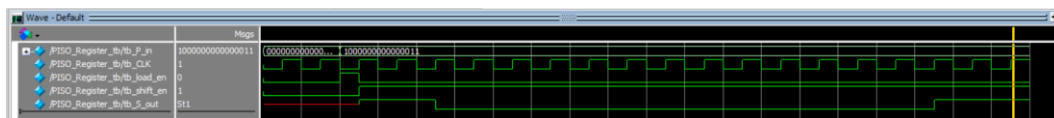


Figure 13. Simulation of 16-Bit Parallel-In, Serial/Parallel-Out Shift Register.

The 4-tap LUT decodes a 4-bit word by selecting the corresponding line in the structure and outputting a coefficient sum pre-computed by the system. Sixteen possible binary linear combinations exist for a 4-bit address entered. The top row represents the dot product of a 4-bit word of zeros with the four coefficients, which results in zero. The same process is applied to all other outputs, up to the sum of all four coefficients.

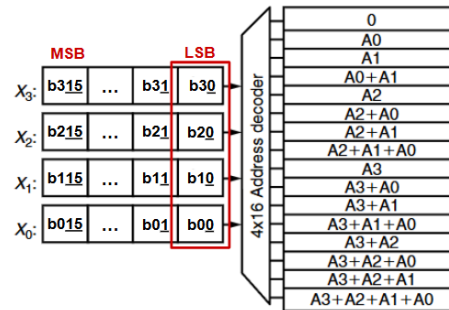


Figure 14. Block Diagram of 4-Tap Look-up Table [4].

The Verilog implementation is structured as a module that uses a 4-bit address and four signed 12-bit filter coefficients to output a signed 14-bit value. The same coefficients used for the standard model are used for the optimized design to align signal-processing behaviors and ensure accurate comparisons of processing capabilities and efficiencies. Bit widths were determined by identifying the sizes of the four highest coefficients and computing the largest possible partial sum. A case statement was chosen to select the coefficient sum corresponding to the 4-bit word, as it has a structure similar to an LUT and is a relatively straightforward design for the number of possible combinations.

```

1 module LUT(
2     input RST,
3     input [3:0] word_in,
4     input signed [11:0] c0, c1, c2, c3, // 1-bit from 4 shift registers (sent out from LSB position)
5     output reg signed [13:0] C_sum // coefficients for filtering
6 );
7
8 always @(word_in) begin
9
10     if (RST == 1'b1) C_sum = 0;
11
12     else begin
13
14         case (word_in) // 4-bit word inputted, (each bit can have a value of 1 or 0, so irrelevant coefficients eliminated)
15
16             4'b0000: C_sum = 0;
17             4'b0001: C_sum = c0;
18             4'b0010: C_sum = c1;
19             4'b0011: C_sum = c1 + c0;
20             4'b0100: C_sum = c2;
21             4'b0101: C_sum = c2 + c0;
22             4'b0110: C_sum = c2 + c1;
23             4'b0111: C_sum = c2 + c1 + c0;
24             4'b1000: C_sum = c3;
25             4'b1001: C_sum = c3 + c0;
26             4'b1010: C_sum = c3 + c1;
27             4'b1011: C_sum = c3 + c1 + c0;
28             4'b1100: C_sum = c3 + c2;
29             4'b1101: C_sum = c3 + c2 + c0;
30             4'b1110: C_sum = c3 + c2 + c1;
31             4'b1111: C_sum = c3 + c2 + c1 + c0;
32             default: C_sum = 14'sd0;
33
34         endcase
35     end
36 end
37 endmodule

```

Figure 15. Verilog Code of 4-Tap Look-up Table.

Arithmetic accuracy across all addresses was verified using ModelSim waves. Coefficients were configured to fixed static values: 10, -20, 30, and -40. A loop was generated by the simulator that indexes a counter from 0 to 16 to verify each binary configuration.

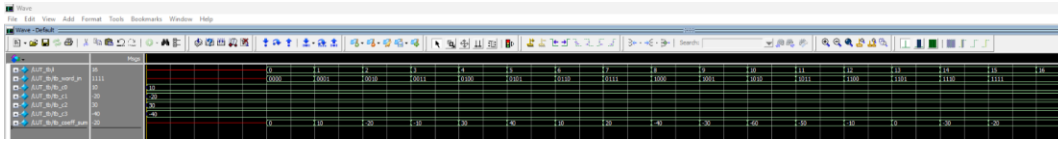


Figure 16. Simulation of 4-Tap Look-up Table.

Designing the PISPO Shift Register and LUT as independent Verilog modules enabled general-purpose reconfigurability, as they lacked same-cycle input/output dependencies. Conversely, the Adder Tree required cascaded logic, and the Accumulator needed only one instance, so both were implemented directly in the top-level hardware description system file.

Finite State Machine

A Finite State Machine (FSM) is a digital design technique used to control when tasks are executed by contributing parts of a system. Moore FSMs rely on control signals sent to components, which are triggered when the corresponding state is entered. A state diagram was developed to model the logic flowchart in terms of the control signals received and sent by an FSM in the top-level design.

The circles indicate the stage of processing a bit, while the arrows show which state will be executed after the current one. Some arrows are labeled with the conditions a control input must satisfy before the system can transition to the next state.

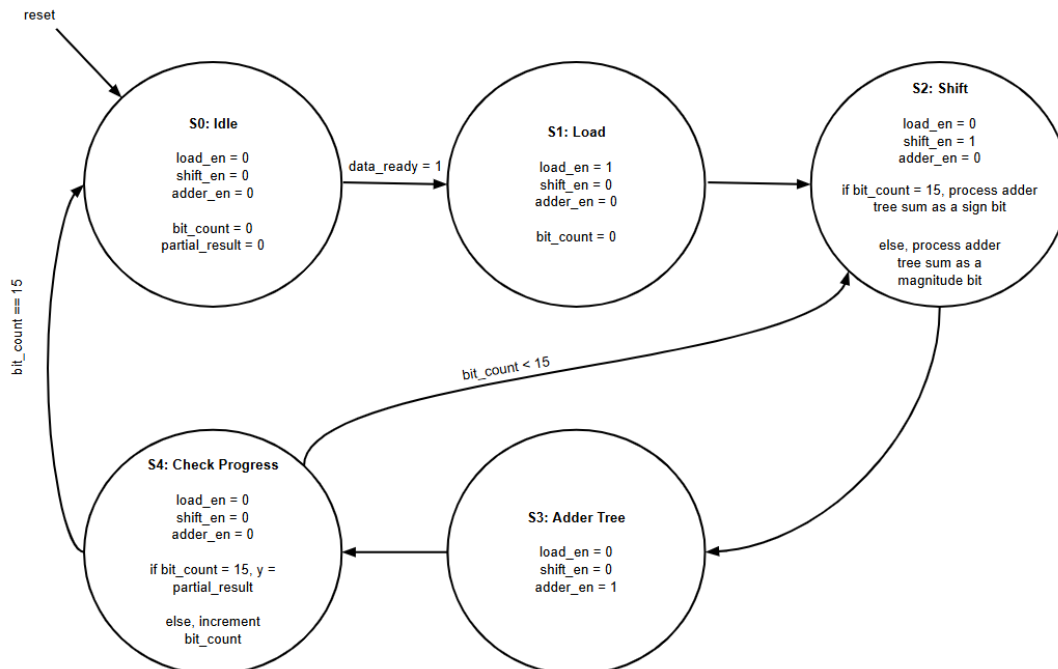


Figure 17. Moore Finite State Machine for Filter Control Logic.

System Assembly

A top-level design was developed to construct the component modules and additional key features into the optimized FIR Filter hardware architecture using the behavioral and structural models as a guide. The system was initially configured to process a single acoustic data point before transitioning to multiple data points. The Verilog design of the system consists of three parts: signal declarations, component instantiations, and control logic.

Input signals control the filter and are triggered externally by the testbench in ModelSim, including a clock for timing updates to values stored in memory, a reset across all system components, a status flag indicating when an entry is ready for processing, and the 16-bit data point. Each resulting filtered data point is output from the system through a 32-bit signal. Additionally, filter coefficients are read locally from a text file and stored in an array.

A Moore FSM encompasses the control logic. States dictate when tasks are executed during a cycle of bit processing and are defined alongside control signals and intermediate output signals to hold values sent from component modules.

```
1 module FIR_Filter_Top(
2     input CLK, // paces memory updates
3     input RST, // clears current/previous results to prepare for processing a new dataset
4     input data_ready, // signal from testbench indicating when ready for filter to process a data point
5     input signed [15:0] data, // raw data point
6     output reg signed [31:0] y // filtered data point
7 );
8
9 // FIR FILTER COEFFICIENTS
10
11
12 reg signed [11:0] coeffs [190:0]; // 191 12-bit coefficients
13
14 initial begin
15     $readmemb("coefficients.txt", coeffs); // read values from MATLAB filter designer application
16 end
17
18 // FSM VARIABLES
19
20
21 // one-hot state encoding
22 parameter S0 = 3'b000, // idle: wait for data to be loaded for processing
23           S1 = 3'b001, // load data into shift registers
24           S2 = 3'b010, // each shift register outputs a bit
25           S3 = 3'b011, // adder tree computes sum of partial weighted contributions
26           S4 = 3'b100; // accumulator adds partial sum to running sum
27
28 reg [2:0] present_state, next_state; // variables to allow system to track when to execute various types of logic
29
30 // sequential logic (memory) outputs
31 reg load_en, shift_en, adder_en; // enable component modules to execute assigned tasks
32 reg [3:0] bit_count; // track amount of bits per data point at a given time
33 reg Ts; // enables computations considering the sign bit
34
35 // combinational logic (tied to control logic) outputs
36 reg load_en_c, shift_en_c, adder_en_c;
37 reg [3:0] bit_count_c;
38 reg [31:0] y_c;
39 reg Ts_c;
40
41 // intermediates
42 wire [190:0] shift_out; // bit sent from PISPO shift register
43 wire signed [13:0] coeff_sum [47:0]; // partial sums from LUTs
44 wire signed [19:0] add_A [23:0]; // sums within levels of adder tree
45 wire signed [19:0] add_B [5:0];
46 wire signed [19:0] add_C [5:0];
47 wire signed [19:0] add_D [2:0];
48 wire signed [19:0] add_E [1:0];
49 wire signed [19:0] add_F; // final sum from adder tree
50
51 reg signed [31:0] partial_result; // accumulated sum of weighted values while processing bits, sequential
52 reg signed [31:0] partial_result_c; // combinational
53
54 wire signed [15:0] PISPO_data_chain [190:0]; // loads data points in parallel into shift registers (1 data point/register)
55 // shifts current data points down when new data point is loaded into 0th index
56
```

Figure 18. Verilog Code of Signal Declarations in Top-Level Design.

The PISPO Shift Register and LUT were designed and validated as isolated components before pursuing top-level system development. Dividing the foundational parts of the system into isolated portions that could be tested independently reduced the risk of troubleshooting design errors that were difficult to identify when the entire system was developed and connected at once before troubleshooting began.

With these components functioning properly independently, the only errors left to identify were misaligned connections during system assembly. Generate constructs are used to instantiate the required number of components for the system to take advantage of repetitive declarations.

Some first and last instantiations differ due to distinct conditions at those boundaries of the architecture. Processing a new data point requires loading it into the first PISPO Shift Register instantiation, whereas subsequent registers process previously filtered values. Additionally, one coefficient input of the final LUT is set to zero since it is unused. There are 192 available spaces across 48 instantiations, but only 191 filter coefficients are needed in this application.

```

62 genvar i; // parallel-in, serial/parallel-out shift registers | 191 instantiations
63 generate
64   for (i = 0; i < 191; i = i + 1) begin : PISPO_GEN // elements of dataset memory are zero if data points are loaded at indices
65     if (i == 0) begin
66       PISPO_Register R (
67         .P_in(data), // load new data point
68         .CLK(CLK),
69         .RST(RST),
70         .load_en(load_en),
71         .shift_en(shift_en),
72         .S_out(shift_out[i]),
73         .P_out(PISPO_data_chain[0]) // load new data point in 0th index of dataset memory
74       );
75     end
76     else begin
77       PISPO_Register R (
78         .P_in(PISPO_data_chain[i-1]), // load previous data points stored in memory
79         .CLK(CLK),
80         .RST(RST),
81         .load_en(load_en),
82         .shift_en(shift_en),
83         .S_out(shift_out[i]),
84         .P_out(PISPO_data_chain[i]) // load previous data points into dataset memory, shifting to higher index
85       );
86     end
87   end
88 endgenerate
89
90 genvar j; // look-up tables | 48 instantiations | generate partial sums based on bit values from data points
91 generate
92   for (j = 0; j < 47; j = j + 1) begin : LUT_GEN
93     LUT lut_inst (
94       .RST(RST),
95       .word_in({shift_out[j*4 + 3], shift_out[j*4 + 2], shift_out[j*4 + 1], shift_out[j*4]}), // load 4 bits sent from PISPO shift register
96       .c0(coeffs[j*4]), // load 4 coefficients from 191 available
97       .c1(coeffs[j*4 + 1]),
98       .c2(coeffs[j*4 + 2]),
99       .c3(coeffs[j*4 + 3]), // store sum in vector
100       .c_sum(coeff_sum[j])
101     );
102   end
103 endgenerate
104
105 LUT lut_47 ( // 1 extra coefficient slot >>> set to zero since not needed
106   .RST(RST),
107   .word_in({1'b0, shift_out[190], shift_out[189], shift_out[188]}),
108   .c0(coeffs[188]),
109   .c1(coeffs[189]),
110   .c2(coeffs[190]),
111   .c3(1'b0), // zero
112   .c_sum(coeff_sum[47])
113 );

```

Figure 19. Verilog Code of Component Instantiations in Top-Level Design.

Similar to previous component instantiations, generate blocks are implemented to repeat computations across four of the six levels in the Adder Tree.

```

119 genvar a;
120 generate // 48 partial sums to 24
121   for (a = 0; a < 24; a = a + 1) begin : LAYER_A // generate loop to repeat 2-input adder computations
122     assign add_A[a] = coeff_sum[a*2] + coeff_sum[a*2 + 1]; // execution of computation
123   end
124 endgenerate
125
126 genvar b;
127 generate // 24 partial sums to 12
128   for (b = 0; b < 12; b = b + 1) begin : LAYER_B
129     assign add_B[b] = add_A[b*2] + add_A[b*2 + 1];
130   end
131 endgenerate
132
133 genvar c;
134 generate // 12 partial sums to 6
135   for (c = 0; c < 6; c = c + 1) begin : LAYER_C
136     assign add_C[c] = add_B[c*2] + add_B[c*2 + 1];
137   end
138 endgenerate
139
140 genvar d;
141 generate // 6 partial sums to 3
142   for (d = 0; d < 3; d = d + 1) begin : LAYER_D
143     assign add_D[d] = add_C[d*2] + add_C[d*2 + 1];
144   end
145 endgenerate
146
147 assign add_E[0] = add_D[0] + add_D[1]; // condense 2 partial sums from add_D into 1 stored in add_E
148 assign add_E[1] = add_D[2]; // pass 3rd value from add_D directly to add_E
149
150 assign add_F = add_E[0] + add_E[1]; // compute sum of 2 final partial sums
151

```

Figure 20. Verilog Code of Adder Tree in Top-Level Design.

Accumulation of partial sums from processing individual bits into a filtered data point is executed within the FSM logic. Values for control signals as well as various outputs related to bit processing and the Accumulator are cleared when the system is reset or otherwise retained in memory.

A combinational system is structured to mirror the conditional state transitions and commands of the state diagram. Control signals are cleared by default to prevent tasks from occurring unprompted, while combinational outputs are updated with the most recent values stored in the sequential logic. The Accumulator determines the filtered data point during State 2 of the FSM. Otherwise, its value is cleared while the system waits to process a new data point.

```

156 always @(posedge CLK) begin
157     if (rst == 1'b1) begin // reset: clear all control signals and outputs
158         present_state <= S0;
159         load_en <= 1'b0;
160         shift_en <= 1'b0;
161         adder_en <= 1'b0;
162         bit_count <= 4'd0;
163         Ts <= 1'b0;
164         partial_result <= 32'sd0;
165     end
166     else begin // store updated values for control signals and outputs in memory
167         next_state <= next_state_c;
168         load_en <= load_en_c;
169         shift_en <= shift_en_c;
170         adder_en <= adder_en_c;
171         bit_count <= bit_count_c;
172         Ts <= Ts_c;
173         partial_result <= partial_result_c;
174         y_c <= y_c;
175     end
176 end
177
178 always @(*) begin
179     // default values
180     load_en_c = 1'b0; // disable tasks component modules can execute
181     shift_en_c = 1'b0;
182     adder_en_c = 1'b0;
183     Ts_c = 1'b0;
184
185     bit_count_c = bit_count; // use values saved in memory for current combinational logic
186     y_c = y;
187     partial_result_c = partial_result;
188     next_state_c = present_state;
189
190     // accumulator
191     if (present_state == S2) begin
192         if (bit_count == 4'd15) begin // MSB (sign bit) cycle
193             partial_result_c = $signed(partial_result) - $signed({12{add_F[19]}}, add_F); // subtract add_F
194         end
195         else begin // LSB cycles
196             partial_result_c = ($signed(partial_result) + $signed({12{add_F[19]}}, add_F)) >> 1; // add add_F then shift result to the right
197         end
198     end
199     else if (present_state == S0) begin // reset variable computing filtered data point if system is in idle state
200         partial_result_c = 32'sd0;
201     end
202 end

```

Figure 21. Verilog Code of Finite State Machine and Accumulator in Top-Level Design.

Case statements are foundational to the FSM state transition logic. Control signals are updated based on the current state, which is structured according to the state diagram. Although not present in the preliminary design, a default setting is included to set the system to State 0, establishing a definitive waiting condition when the process status is uncertain.

```

203 case (present_state)
204     S0: begin // idle
205         if (data_ready == 1'b1) next_state = S1; // transition to loading shift registers when data is ready
206         bit_count_c = 4'd0; // reset bit_count---no data points to process at this time
207     end
208     S1: begin // load data into shift registers
209         next_state = S2; // transition to bit processing after all tasks in this state are complete
210         load_en_c = 1'b1; // enable task for shift registers
211         bit_count_c = 4'd0; // reset bit_count---new data point being processed
212     end
213     S2: begin // shift bits out of shift registers
214         next_state = S3; // transition to accumulation after all tasks in this state are complete
215         shift_en_c = 1'b1; // enable task for shift registers
216     end
217     S3: begin // combine partial sums from processing
218         next_state = S4; // transition to checking bit processing progress after all tasks in this state are complete
219         shift_en_c = 1'b0; // disable shifting task for shift registers
220         adder_en_c = 1'b1; // enable task for adder tree
221     end
222     S4: begin // check bit processing progress
223         adder_en_c = 1'b0; // disable task for adder tree
224
225         if (bit_count == 4'd15) begin // after last bit of data point is processed
226             next_state = S0; // wait for next data point to get loaded
227             shift_en_c = 1'b0; // disable shifting task for shift registers
228
229             y_c = $signed(partial_result_c); // update output variable with value of filtered data point
230         end
231         else begin // when more bits need to be processed in data point
232             next_state = S2; // transition to processing next set of bits after all tasks in this state are complete
233             shift_en_c = 1'b0; // disable shifting task for shift registers
234             bit_count_c = bit_count + 1'b1; // increment counter tracking amount of bits processed
235         end
236     end
237     default: next_state = S0; // automatically set state to idle when status of current/next state is uncertain
238 endcase

```

Figure 22. Verilog Code of State Transition Logic in Top-Level Design.

Verification of General FIR Filter Behavior

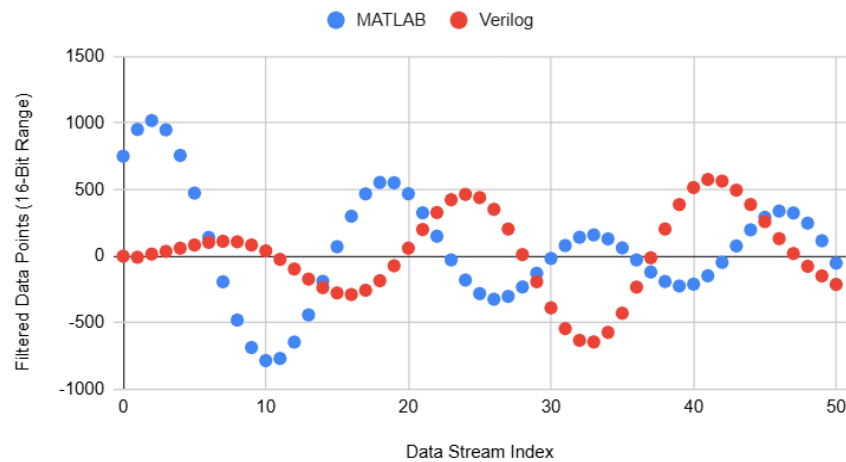


Figure 25. Graph of Filtered Data From Reference Model and Optimized Design.

Unlike the standard model, the optimized filter required 191 initial samples to reach steady-state operation. Testing with a sample dataset produced oscillations similar to the MATLAB reference model, validating core system functionality before scaling to larger datasets.

Design Comparison

The primary motivation for replacing MAC units with DA units in an FIR filter design was to reduce power and resource consumption while maintaining filtering quality. Metrics evaluating these hardware features were estimated using Quartus Prime analysis tools, while MATLAB was utilized to generate time- and frequency-domain visualizations. These evaluations sought to quantify the operational trade-offs of the DA-based architecture relative to the standard MAC-based model.

Standard vs Optimized Finite Impulse Response Filter Comparison

Time-domain analysis was conducted in MATLAB to compare the processing capabilities of the standard MAC and optimized DA models. A mean-square error (MSE) of 1984.77 was calculated between the two outputs, representing a minor variance within the 32-bit integer scale. Additionally, a signal-to-error ratio (SER) of 24.86 dB was observed, confirming minimal quantization errors introduced by the DA framework.

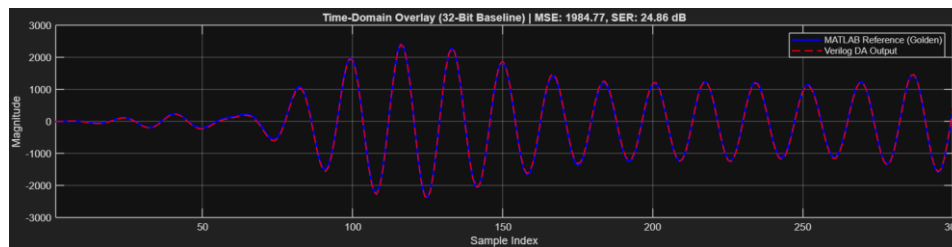


Figure 26. Time-Domain Plot of Results From Standard Filter Overlaid With Optimized Filter.

PSD plots were generated to verify passband isolation across the targeted 10-13 kHz range. The optimized filter demonstrated greater attenuation of stopband frequencies, indicating stronger frequency-rejection capabilities.

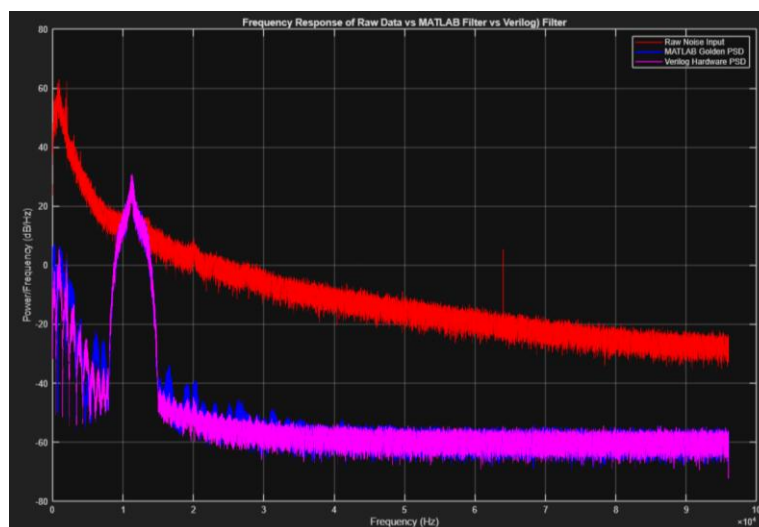


Figure 27. PSD of Raw Signal Overlaid With Results From Both Filters.

To assess hardware efficiency under identical test conditions, a standard filter was generated using the Quartus FIR II IP Parameter Editor. Because the IP generator was restricted to a 28-bit output, the DA-based design was modified to a 28-bit output for equivalent hardware evaluation.

Power metrics were extracted from the Quartus Power Analyzer Summary. Reductions were recorded across all measured categories for the DA-based design. A 72.54% reduction in core dynamic power was achieved by replacing 18x18 multipliers and M9K memory with precomputed lookup tables and shift-accumulator logic, drastically reducing sample-switching activity. I/O power was reduced by 42.33% due to decreased internal routing congestion and reduced memory/DSP bus toggling. Core static power remained nearly unchanged (0.09% difference), as static dissipation comes from the FPGA rather than microarchitecture. These savings contributed to a 14.25% reduction in total thermal power.

Power Dissipation Metric	MAC-Based Filter	DA-Based Filter	Power Reduction
Total Thermal Power	123.26 mW	105.69 mW	14.25%
Core Dynamic Thermal Power	11.36 mW	3.12 mW	72.54%
Core Static Thermal Power	90.05 mW	89.97 mW	0.09%
I/O Thermal Power	21.85 mW	12.60 mW	42.33%

Figure 28. Power Comparison Between Both Filters.

Hardware usage was compiled from the Quartus Fitter Resource Utilization Summary. All dedicated DSP and RAM blocks were eliminated in the DA-based implementation. To replace these dedicated units, lookup tables and adder trees were synthesized using configurable general-purpose components, increasing total logic cell utilization.

Resource Metric	MAC-Based Filter	DA-Based Filter
Logic Cells	67	138
Dedicated Logic Registers	67	67
Memory Bits	2292	0
M9K Blocks	1	0
Digital Signal Processing 18x18 Blocks	1	0
LUT-Only Logic Cells	0	69
Register-Only Logic Cells	66	23
LUT/Register Logic Cells	1	46

Figure 29. Resource Comparison Between Both Filters.

The hardware and signal analysis demonstrated that the DA-based FIR filter provides a power- and resource-efficient alternative to traditional MAC architectures. This design offers high utility for deployment in energy-constrained sensing and aerodynamic data processing applications.

Optimized Finite Impulse Response Filter Output Resolution

The impact of word-length shortening was evaluated by scaling the DA-based filter output from the baseline 32-bit resolution down to a standard 24-bit audio resolution.

Time-domain waveforms were plotted against the standard MAC-based model. The 24-bit DA-based outputs were scaled to 32-bits in MATLAB to facilitate direct signal comparison. The reduced-resolution signal tracked the rising and falling edges accurately, though quantization errors were observed near signal peaks and troughs.

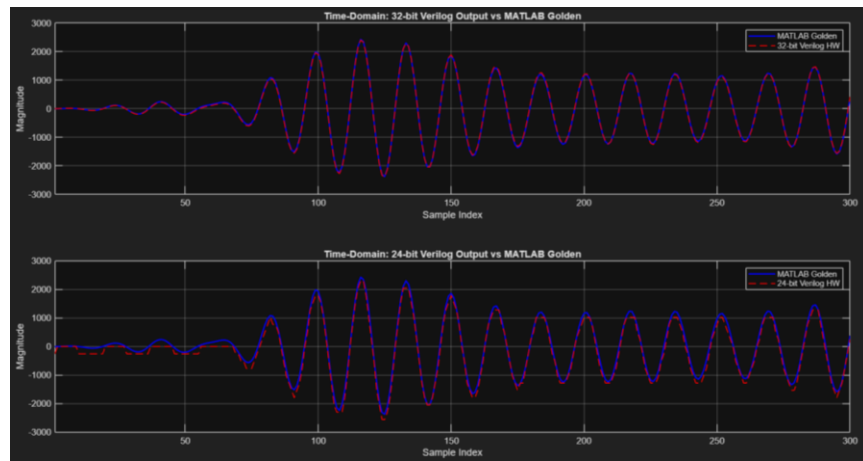


Figure 30. Time-Domain Plots of Results From Standard Filter Overlaid With Optimized Filters.

Moving to 24-bit resolution increased the MSE to 21060.753 and decreased the SER to 14.60 dB. However, phase response and general waveform geometry were preserved.

Resolution Metrics	24-Bit Output Filter	32-Bit Output Filter
Scaling	2^8	2^0
Mean Square Error	21060.753	1984.774
Signal-to-Error Ratio (dB)	14.6	24.86

Figure 31. Filtered Signal Quality Comparison Between Both Optimized Filters.

PSD analysis confirmed that while the 24-bit filter effectively isolated the 10–13 kHz passband, more noise outside the targeted range was present.

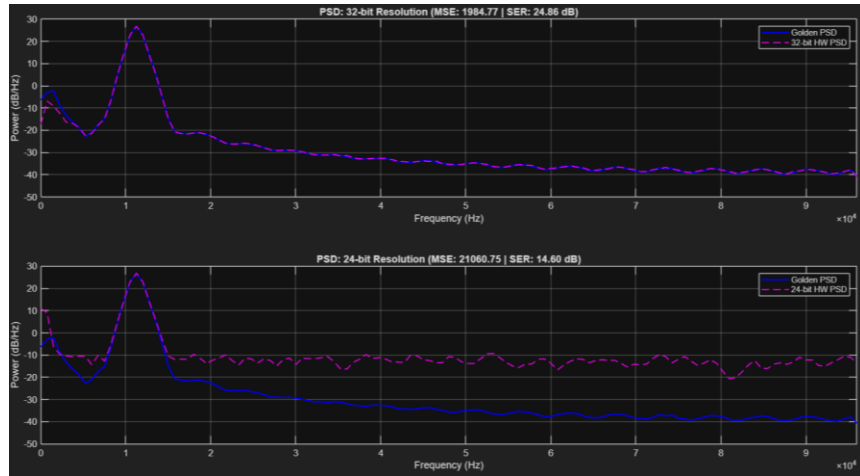


Figure 32. PSD of Results From Standard Filter Overlaid With Optimized Filters.

Transitioning to 24 bits further reduced core dynamic power by 18.63% and I/O power by 7.86%. These factors led to a 1.45% decrease in total thermal dissipation relative to the 32-bit DA-based filter. Logic cell usage was also reduced from 144 to 112 cells.

Power Dissipation Metric	24-Bit Output	32-Bit Output	Power Reduction
Total Thermal Power	104.19 mW	105.72 mW	1.45%
Core Dynamic Thermal Power	2.14 mW	2.63 mW	18.63%
Core Static Thermal Power	89.97 mW	89.97 mW	0%
I/O Thermal Power	12.08 mW	13.11 mW	7.86%

Figure 33. Power Comparison Between Both Optimized Filters.

Resource Metric	24-Bit Output FIR Filter	32-Bit Output FIR Filter
Logic Cells	112	144
Dedicated Logic Registers	55	71
LUT-Only Logic Cells	56	71
Register-Only Logic Cells	23	25
LUT/Register Logic Cells	33	48

Figure 34. Resource Comparison Between Both Optimized Filters.

The 24-bit DA-based filter proved to be the most power- and resource-efficient configuration. However, a design trade-off of a minor reduction in signal-to-noise ratio was exchanged for reduced hardware footprint and dynamic energy dissipation.

References

- [1] R. Amirtharajah, "Distributed Arithmetic," in *Reconfigurable Computing: The Theory and Practice of FPGA-Based Computation*, S. Hauck and A. DeHon, Eds. Burlington, MA, USA: Morgan Kaufmann, 2008, ch. 24, pp. 503–512.
- [2] AMD, "Systolic Multiply-Accumulate," *FIR Compiler v7.2 LogiCORE IP Product Guide (PG149)*. Accessed: Jul. 28, 2026. [Online]. Available: <https://docs.amd.com/r/en-US/pg149-fir-compiler/Systolic-Multiply-Accumulate>.
- [3] M. Serrano, V. Romo, R. Karmarkar, J. Garzon, H. Poole, I. Mo, S. Amezcua, R. Amirtharajah, B. Linke, C. Badrya, "Sensing Flow Conditions on Lifting Surfaces," submitted for publication.
- [4] Hardware architecture symbols adapted from Figure 24.1 [1].



Reflection

Participation in the BE-STARSE Program motivated me to take the initiative to develop an optimized Finite Impulse Response (FIR) filter. Throughout this project, I learned to plan experiments tailored to anticipated testing conditions and rigorously document test configurations to ensure full repeatability. By synthesizing insights from published literature, real-world data behaviors, and hardware constraints, I successfully formulated a robust system architecture. Approaching conceptualization, design, and verification through phased milestones helped clarify my vision for the system while isolating verification and troubleshooting steps.

Building on Phase 1 components, I completed the full design in Phase 2. Comparing the optimized implementation against the traditional design yielded critical insights into the trade-offs between early hardware-level preprocessing and downstream software data analysis. Furthermore, the resulting power and resource efficiencies directly support Next Level Research Green Aviation's sustainability goals in studying airfoil surface geometries and aerodynamic interactions. Evaluating these complex architectural trade-offs significantly strengthened my engineering design mindset for future projects.

I plan to pursue higher education to deepen my expertise in analog and digital systems for hardware development in industry applications. Executing this project provided an invaluable opportunity to apply foundational digital system design principles to a complex engineering challenge, building both my technical confidence and enthusiasm for advancing in these converging fields.



Acknowledgments

Platforms

The following tools and applications were used to execute research for this project.

- Audacity
- MATLAB Filter Designer App and Editor
- Google Slides
- Quartus Prime (v.19.1) Editor and FIR II IP Parameter Editor
- ModelSim

Artificial Intelligence

Google Gemini was used as an AI-assisted writing tool for portions of this report, specifically to refine prose readability. All AI-generated suggestions were reviewed, verified, and edited to ensure technical accuracy and alignment with the UC Davis Acceptable Use Policy.

Mentors & Sponsors

I want to express my sincere gratitude to Professor Saif Islam and Professor Zhi Ding for organizing the BE-STARSE Program and making this opportunity possible. I am especially indebted to Professor Rajeevan Amirtharajah and the members of his research group, whose mentorship and continuous guidance inspired my work throughout this project. I also extend my thanks to Professor Barbara Linke and Professor Camli Badrya, as well as their lab members, for welcoming me into their research groups during my lab rotations. Lastly, I gratefully acknowledge the National Science Foundation for funding this career development opportunity.



Faculty Mentor Approval

Rajeevan Amirtharajah

Name

August 19, 2026

Date

This material is based upon work supported by the **National Science Foundation (NSF)** under grant number **2347866**.